物件導向程式設計 2

A1143305 吳宥穎

U1_1 骰子模擬器

AI 對話紀錄



幫我用 Java 寫一個這個的程式碼，要可以執行。

這是一份完全符合題目規格、嚴謹且可以直接在 Java 環境（如 VS Code、Eclipse 或線上編譯器）中執行的 GUI 程式碼。 PNG

Java 程式碼（可直接執行）

Java

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
```

```java
public class Main extends JFrame {
    // 統計數據變數
    private int count = 0;       // 擲骰總次數 N
    private int sum = 0;         // 點數總和 M
    private Random random;

    // UI 元件
    private JLabel infoLabel;    // 上方統計資訊標籤
    private JLabel diceLabel;    // 中央點數顯示標籤
    private JButton rollButton;  // 下方擲骰子按鈕

    public Main() {
        // 1. 初始化視窗 basic 設定
        setTitle("骰子模擬器");                    // 視窗標題
        setSize(400, 320);                        // 尺寸 400x320
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 關閉時結束程式
        setLocationRelativeTo(null);              // 開啟時置中
        setLayout(new BorderLayout());

        random = new Random();

        // 2. 視窗上方顯示「已擲 N 次，總和 M，平均 X.XX」
```



這些部分看不太懂。

這幾張圖片截圖中的程式碼，是我們剛才幫你寫的 Java Swing **視窗程式（骰子模擬器）** 中的核心邏輯！ PNG

我用最白話、最直覺的方式，把這 4 張圖對應的功能與語法一行行拆解給你聽：

第一張圖：建立與設定「上方統計文字」

Java

第一張圖：建立與設定「上方統計文字」

第一張圖：建立與設定「上方統計文字」

```java
infoLabel = new JLabel("已擲 0 次，總和 0，平均 0.00", SwingConstants.CENTER);
infoLabel.setFont(new Font("SansSerif", Font.PLAIN, 16));
infoLabel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
add(infoLabel, BorderLayout.NORTH);
```

- `new JLabel(...)`：在視窗裡放一塊文字標籤。預設顯示 `已擲 0 次...`，並用 `SwingConstants.CENTER` 讓文字在標籤內**水平居中**。
- `setFont(...)`：設定字型風格（無襯線字型）、普通體（`PLAIN`），字體大小為 `16` pt。
- `setBorder(...)`：設定外邊框間距（上、左、下、右各留 10 像素的空白），讓文字不會死死黏在視窗頂端邊框，比較好看。

第二張圖：按鈕事件監聽器（當按鈕被按下去時要幹嘛）

```java
rollButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        rollDice();
    }
});
```

- `addActionListener(...)`：給按鈕裝上一個「監聽器」，隨時監控使用者有沒有點擊這個按鈕。
- `@Override public void actionPerformed(...)`：當使用者**真的點了這個按鈕**，Java 就會自動跳進來執行這裡面的程式碼。
- `rollDice()`：當按鈕被點擊，立刻呼叫我們自己寫好的 `rollDice()`（擲骰子）方法去

Reflection

AI 給的哪裡有問題

- AI 初期輸出的程式碼在每一行都加上了極為詳細甚至冗長的行內註解，雖然有助於逐行理解，但在需要乾淨交作業時顯得繁雜。

- 不了解程式最上方的工具欄（如 import 引用模組與元件套件）、按鈕事件監聽器、擲骰子的數學與資料更新、改變文字顏色與格式化更新統計數據。

改了什麼？

- 要求 AI 移除所有行內註解（Clean Code），還原出乾淨且結構明確的 Java Swing 程式碼。

- 請 AI 用最簡單直覺的方式，把視窗頂端佈局 (infoLabel)、按鈕事件監聽器 (ActionListener)、隨機數生成 (random.nextInt(6) + 1) 與平均值計算，以及 String.format 格式化與顏色判斷 (setForeground) 再逐區拆解解釋一遍。

為什麼要這樣改？

- 過多的註解會遮蔽真正的程式邏輯；清空註解並維持命名規則，能讓程式碼更符合實際開發中的乾淨樣貌。

- 唯有將 Swing GUI 的基礎語法與事件處理機制真正釐清，才不會停留於盲目複製，並確保後續面對題目變形或上機考時能夠獨立編寫程式。